CE-2103 · Lección 03

Análisis de Algoritmos

Medir la eficiencia en tiempo y espacio

analisis.java

```java
// ¿Cuánto crece el tiempo?
for (int i = 0; i < n; i++)
    foo(v[i]);
// f(n) = 6nT + T = O(n)
```

Instituto Tecnológico de Costa Rica · Ingeniería en Computadores

¿Qué aprenderemos hoy?

1. Definir algoritmo y sus características
2. Comparar el análisis empírico y el teórico
3. Distinguir el mejor, el peor y el caso promedio
4. Usar las notaciones Big O, Ω (Omega) y Θ (Theta)
5. Determinar la complejidad de loops y de recursión
6. Resolver recurrencias y aplicar el análisis amortizado

// algoritmos

Algoritmo: un procedimiento para cumplir una tarea

1

Input

Un problema bien definido. En la receta: los ingredientes.

2

Algoritmo

Las instrucciones que transforman la entrada. En la receta: los pasos de cocina.

3

Output

El resultado deseado. En la receta: la comida.

¿Programa vs. algoritmo? un programa es la implementación de un algoritmo en un lenguaje de programación.

// algoritmos

Características de un algoritmo

1. **No es ambiguo**
   Cada paso tiene un solo significado.
2. **Entrada bien definida**
   Debe ser clara y consistente.
3. **Salida bien definida**
   Indica qué tipo de output genera.
4. **Es finito**
   Termina en un tiempo determinado.
5. **Es factible**
   Se puede ejecutar con los recursos tecnológicos disponibles.

Representaciones de un algoritmo

Pseudocódigo

```
LEER n
p ← 1
MIENTRAS n ≠ 0 HACER
    p ← p · n
    n ← n − 1
FIN MIENTRAS
RETORNAR p
```

Lenguaje natural

«El factorial de un entero no negativo n, denotado n!, es el producto de todos los enteros positivos menores o iguales que n.»

Definición formal

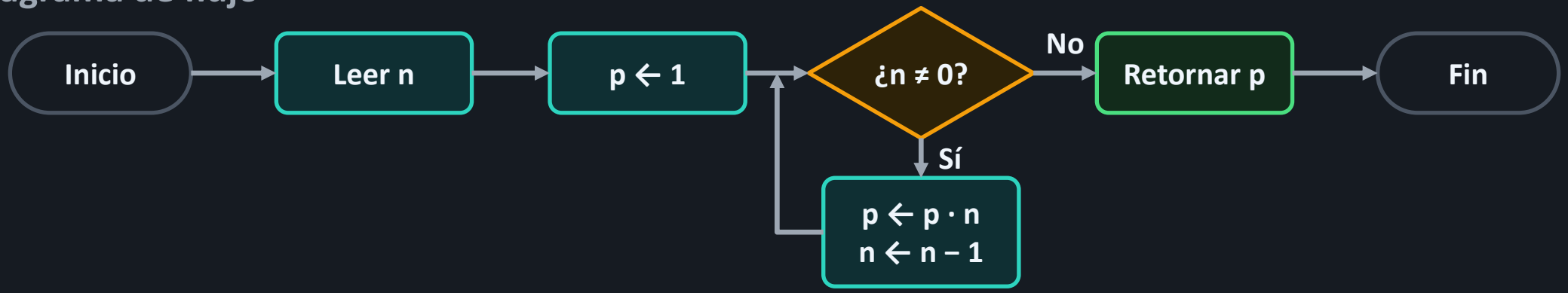
factorial : N → N

Para todo i en N:

factorial(0) = 1

factorial(i + 1) = (i + 1) · factorial(i)

Diagrama de flujo



// análisis

¿Por qué analizar un algoritmo?

Analizar es determinar su **eficiencia** en tiempo y espacio. Sirve para:

Predecir

El comportamiento del algoritmo y del programa que lo implementa.

Comparar

Distintos algoritmos que sirven para el mismo propósito.

Optimizar

Conociendo el comportamiento, se puede mejorar.

Clasificar

El algoritmo según su complejidad.

> **Muchos factores influyen:** hardware, sistema operativo, lenguaje, cantidad de datos y hasta la pericia del programador.

Análisis empírico: benchmarking

1. Marcar un tiempo de inicio
2. Ejecutar el programa con un input dado
3. Marcar el tiempo final
4. Calcular el tiempo transcurrido
5. Repetir la prueba varias veces y calcular la mediana

// análisis empírico

Profiling y el límite del análisis empírico

Profiling

Es análisis empírico: técnica dinámica para hallar cuellos de botella (tiempo, memoria, CPU).

Muy usado

Es común en la práctica: hay ambientes controlados donde se puede medir.

Depende de la máquina

No se puede generalizar: una máquina de hace 10 años no es una moderna.

Necesitamos un análisis que **no dependa de la máquina**: el análisis teórico.

El modelo computacional del análisis teórico

1

Operaciones simples

+, -, *, /, =, == toman tiempo constante: las operaciones elementales de Turing.

2

Loops y funciones

No son operaciones simples: son la composición de muchas operaciones simples.

3

Acceso a memoria

Cada acceso a memoria toma tiempo constante.

Se mide `f(n)`: pasos (tiempo) o posiciones de memoria (espacio) según el tamaño n. No importan los nanosegundos, sino la **tasa de crecimiento**: el algoritmo es **eficiente** si `f(n)` crece lentamente.

// análisis teórico

Contar operaciones: el máximo de un arreglo

Peor caso: el if siempre se cumple (T = costo de una operación simple)

```
int max(int[] array) {
    int max = array[0];                          // 3T
    for (int i = 1; i < array.length; i++) {     // T + 2nT
        if (array[i] > max) {                    // 2T(n-1)
            max = array[i];                      // 2T(n-1)
        }
    }
    return max;                                  // T
}
```

Sumando las líneas

$3T + (T + 2nT) + 2T(n-1) + 2T(n-1) + T$

$$f(n) = 6nT + T$$

Ojo: este nivel de detalle no es práctico. Más adelante lo simplificamos con el **análisis asintótico**.

Empírico vs. teórico

Análisis empírico

Consiste en ejecutar un programa, un conjunto de programas u otras operaciones para evaluar el desempeño relativo de un objeto, normalmente con una serie de pruebas estándar. También se conoce como **benchmark**.

Ventaja	Muy fácil de entender y calcular.
Desventaja	Los resultados pueden variar según el hardware.
Desventaja	Requiere ejecutar el código.

Análisis teórico de algoritmos

Es una parte importante de la teoría de complejidad computacional: estima los recursos que necesita un algoritmo para resolver un problema específico.

Ventaja	Independiente de la plataforma: predice mejor el desempeño.
Desventaja	Difícil de calcular y de entender.
Ventaja	Se calcula con solo mirar el código.

Empírico: depende del hardware · Teórico: independiente de la plataforma.

// mejor, peor y promedio

Mejor, peor y caso promedio

Mejor caso

Se realiza la menor cantidad de operaciones. Ej.: el elemento buscado es el primero.

Peor caso

Se realiza la mayor cantidad de operaciones. Ej.: el elemento buscado es el último.

Caso promedio

Valor esperado según la probabilidad de cada input. Ej.: el elemento está en cualquier posición.

El **peor caso** es el más útil: es una cota superior fácil de calcular (aunque a veces pesimista). Si es aceptable, el algoritmo es eficiente.

// mejor, peor y promedio

Búsqueda secuencial: el algoritmo y su mejor caso

Mejor caso: x está en a[0] (T = costo de una operación simple)

```
boolean contains(int[] a, int x) {
    for (int i = 0; i < a.length; i++) {  // 2T
        if (x == a[i]) {                  // 2T
            return true;                  // T
        }
    }
    return false;
}
```

Se retorna en la primera vuelta: no se ejecuta `i++`.

Mejor caso

x está en `a[0]`: solo se ejecuta la primera vuelta.

2T + 2T + T = 5T

Peor caso

x no está (o es el último): se revisan los n elementos.

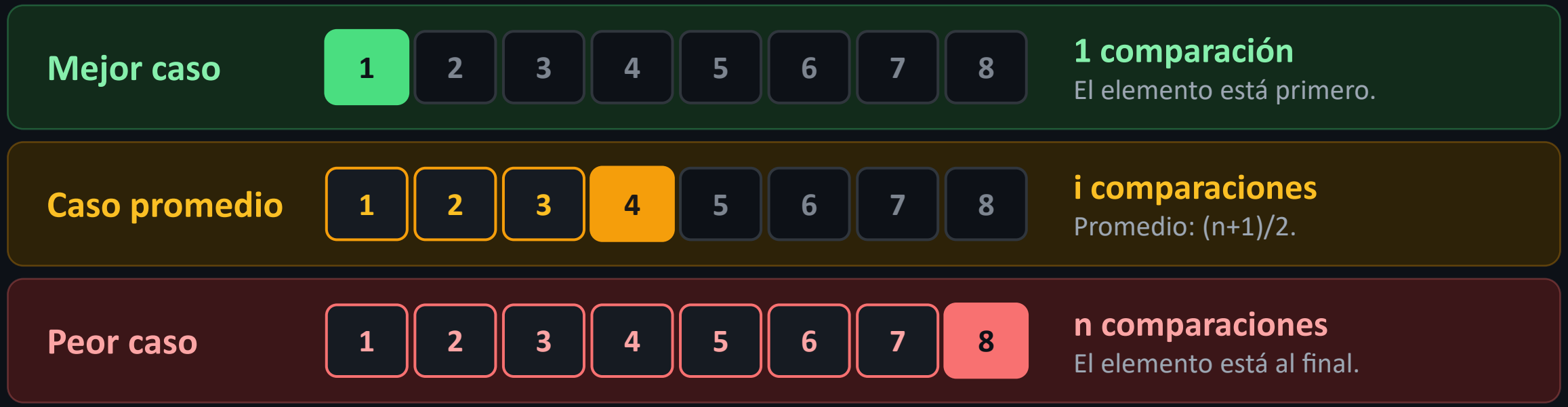
n comparaciones

Constante: el mejor caso cuesta 5T aunque el arreglo tenga un millón de elementos: **no depende de n**.

// mejor, peor y promedio

Búsqueda secuencial: los tres casos

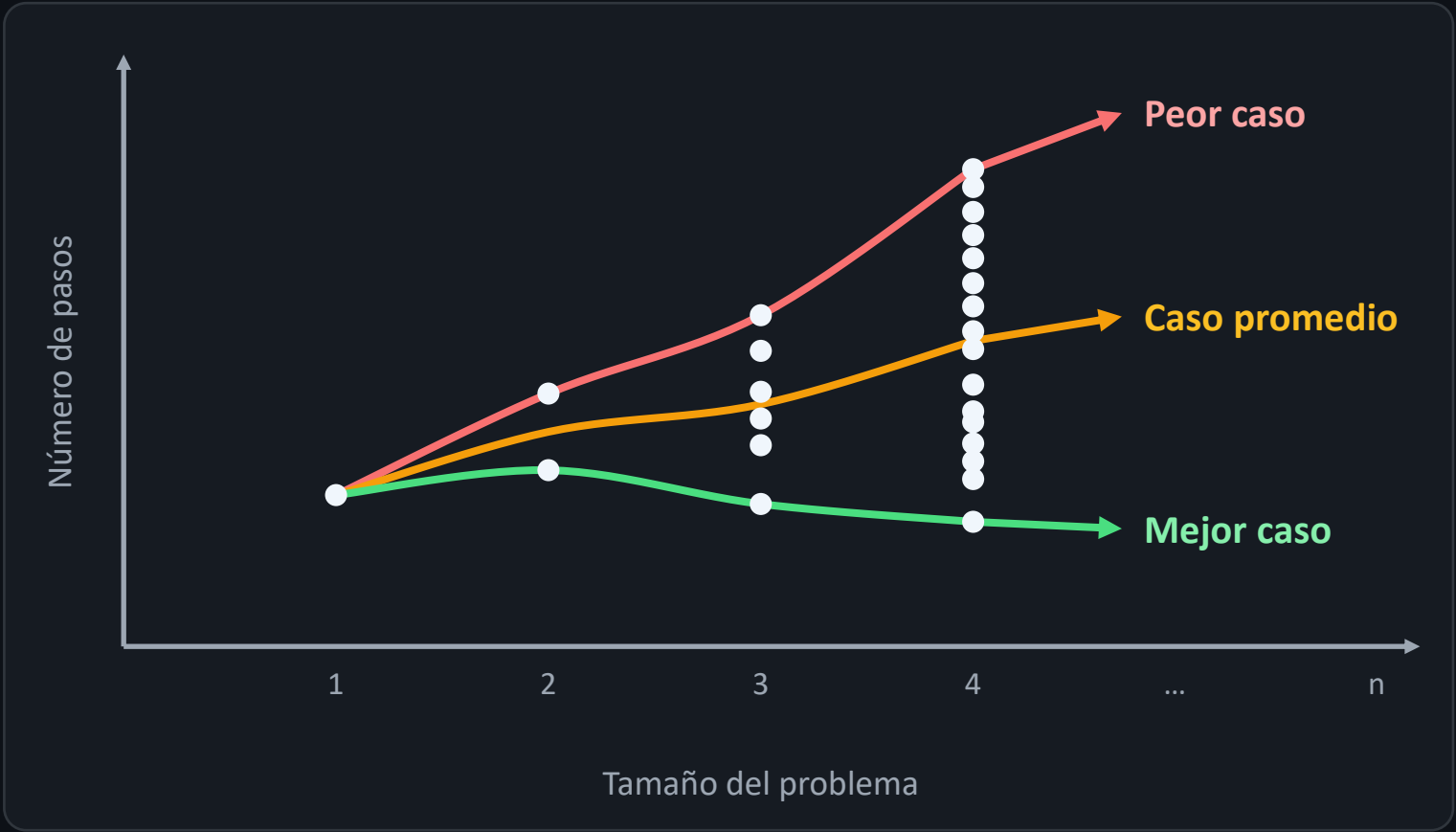
Arreglo de n = 8 posiciones: cada celda revisada es una comparación; la celda sólida es el elemento buscado.



Promedio: si el elemento está y cada posición es igual de probable (`p = 1/n`): `1 + 2 + ... + n = n(n+1)/2`; al dividir entre n queda `(n+1)/2`.

// mejor, peor y promedio

Pasos vs. tamaño del problema



Peor caso
Máximo de pasos entre las entradas de tamaño n.

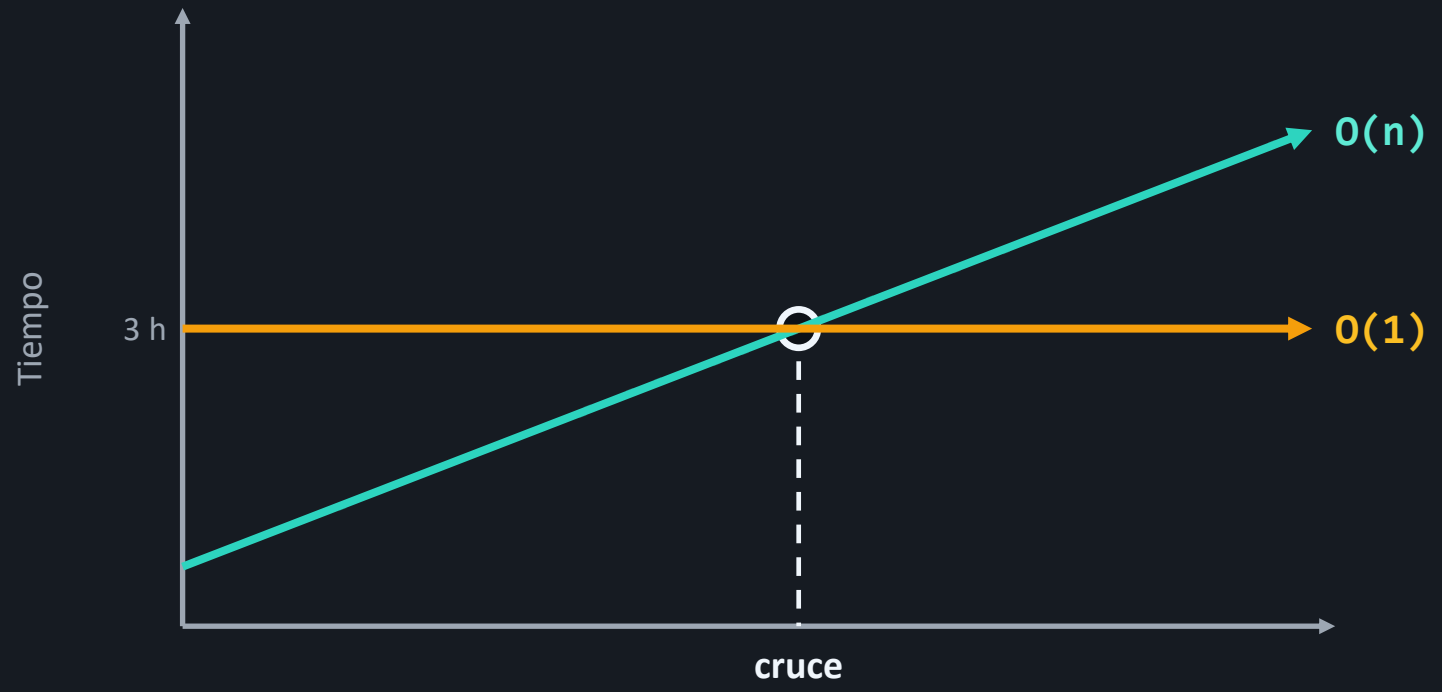
Caso promedio
Promedio de pasos de esas entradas.

Mejor caso
Mínimo de pasos entre esas entradas.

Cada punto
Una entrada posible de ese tamaño.

// análisis asintótico

¿Correo/FTP o llevarlo a Guanacaste?



Por red: O(n)
El tiempo crece con el tamaño del archivo.

En persona: O(1)
Siempre son 3 h, sin importar el tamaño.

Cruce
Desde aquí, llevarlo en persona es más rápido.

¿Qué término determina el crecimiento?

Si contar los pasos de un código da `f(n) = 2n^3 + 3n^2 + log10(n) + 333`

n	2n^3	3n^2	log10(n)	Constante	f(n)	2n^3 / f(n)
1	**2**	3	0	333	338	**0.59 %**
10	**2 000**	300	1	333	2 634	**75.93 %**
100	**2 000 000**	30 000	2	333	2 030 335	**98.51 %**
1 000	**2 000 000 000**	3 000 000	3	333	2 003 000 336	**99.85 %**
10 000	**2 000 000 000 000**	300 000 000	4	333	2 000 300 000 337	**99.99 %**

Al crecer n, **2n^3** casi lo es todo: los demás términos y la constante dejan de importar. Por eso `f(n) = O(n^3)`.

Análisis asintótico: descartar lo menos relevante

Función	Notación asintótica
f(n) = 12754n^2 + 4353n + 834 lg2 n + 13546	O(n^2)
f(n) = 6nT + T	O(n)
O(n^2 + n)	O(n^2)
O(n + log(n))	O(n)
O(5 * 2^n + 100n^2)	O(2^n)

Se descartan los **términos de menor relevancia** y las **constantes**: lo que importa es la **escalabilidad**.

// análisis asintótico

Un loop lineal: O(n)

Recorrido de un vector

```
for (int i = 0; i < vector.length(); i++) {
    foo(vector[i]); // foo: tiempo constante
}
```

Complejidad

Trabajo constante por cada elemento del vector.

O(n)



Si cambia la constante, cambia la pendiente, pero la **tendencia** sigue siendo lineal.

// notaciones

Big O, Big Omega y Big Theta

Big O

Límite superior: el algoritmo no es más lento que esta cota.

Ej.: O(n^2)

Big Omega

Límite inferior: el algoritmo es al menos tan lento como esta cota.

Ej.: Ω(n^2)

Big Theta

Comportamiento exacto: el límite ajustado (O y Ω a la vez).

Ej.: Θ(n^2)

Ojo: el mejor, el peor y el promedio se pueden describir con **cualquiera** de las tres. Big O no es «el peor caso».

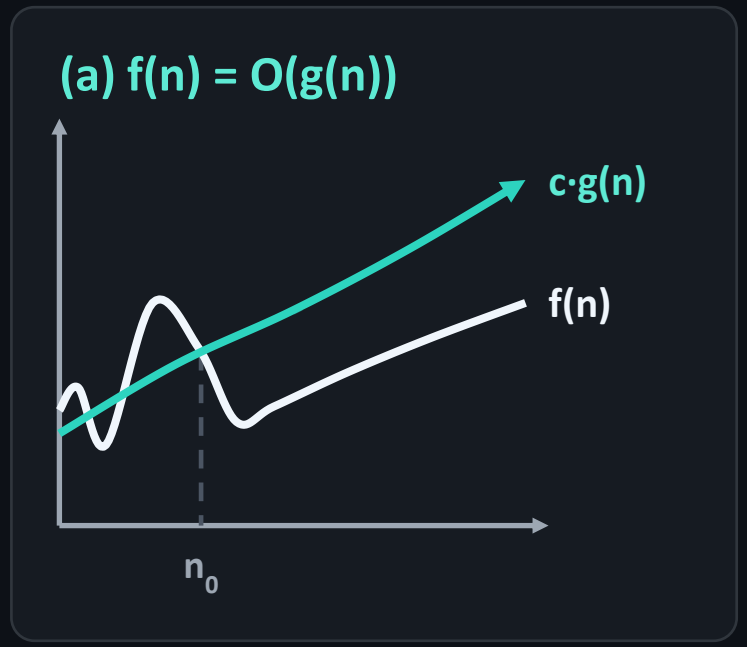
// notaciones

Big O, Ω y Θ: definición formal

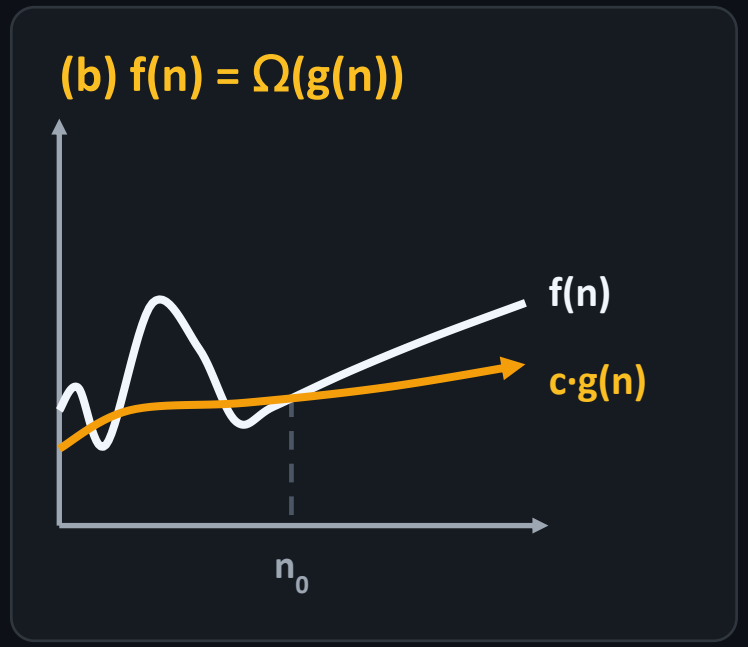
Notación	Cota	Definición (para todo n > n0)	Ejemplo: f(n) = 3n^2 - 100n + 6
Big O	Superior	f(n) <= c · g(n)	c = 3 y g(n) = n^2: 3n^2 siempre es mayor → O(n^2)
Big Omega	Inferior	f(n) >= c · g(n)	c = 2: 2n^2 < f(n) si n > 100 → Ω(n^2)
Big Theta	Ajustada	c1 · g(n) <= f(n) <= c2 · g(n)	Aplican O(n^2) y Ω(n^2) → Θ(n^2)

`g(n)` es el intento de clasificar a `f(n)`; las constantes `c`, `c1` y `c2` las podemos escoger.

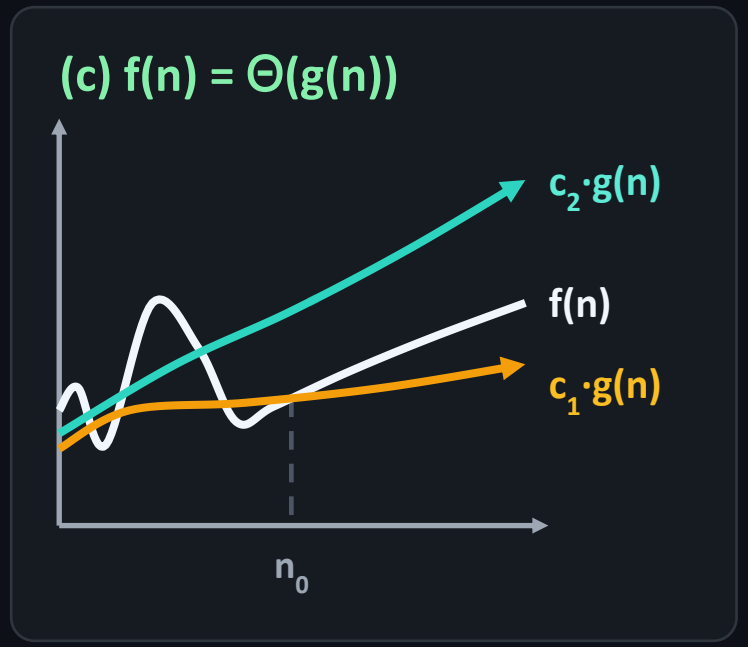
Visualización de O, Ω y Θ



Cota superior: desde n_0, c·g(n) queda por encima de f(n).



Cota inferior: desde n_0, c·g(n) queda por debajo de f(n).



Cota ajustada: desde n_0, f(n) queda entre $c_1 \cdot g(n)$ y $c_2 \cdot g(n)$.

Lo que pasa antes de `n0` no importa: solo interesa el comportamiento para **n grande**.

// complejidad espacial

Complejidad espacial: la memoria que se necesita

Recursión

```
int sum(int n) {
    if (n <= 0) {
        return 0;
    }
    return n + sum(n-1);
}
```

¿Cuál es la complejidad espacial de este?

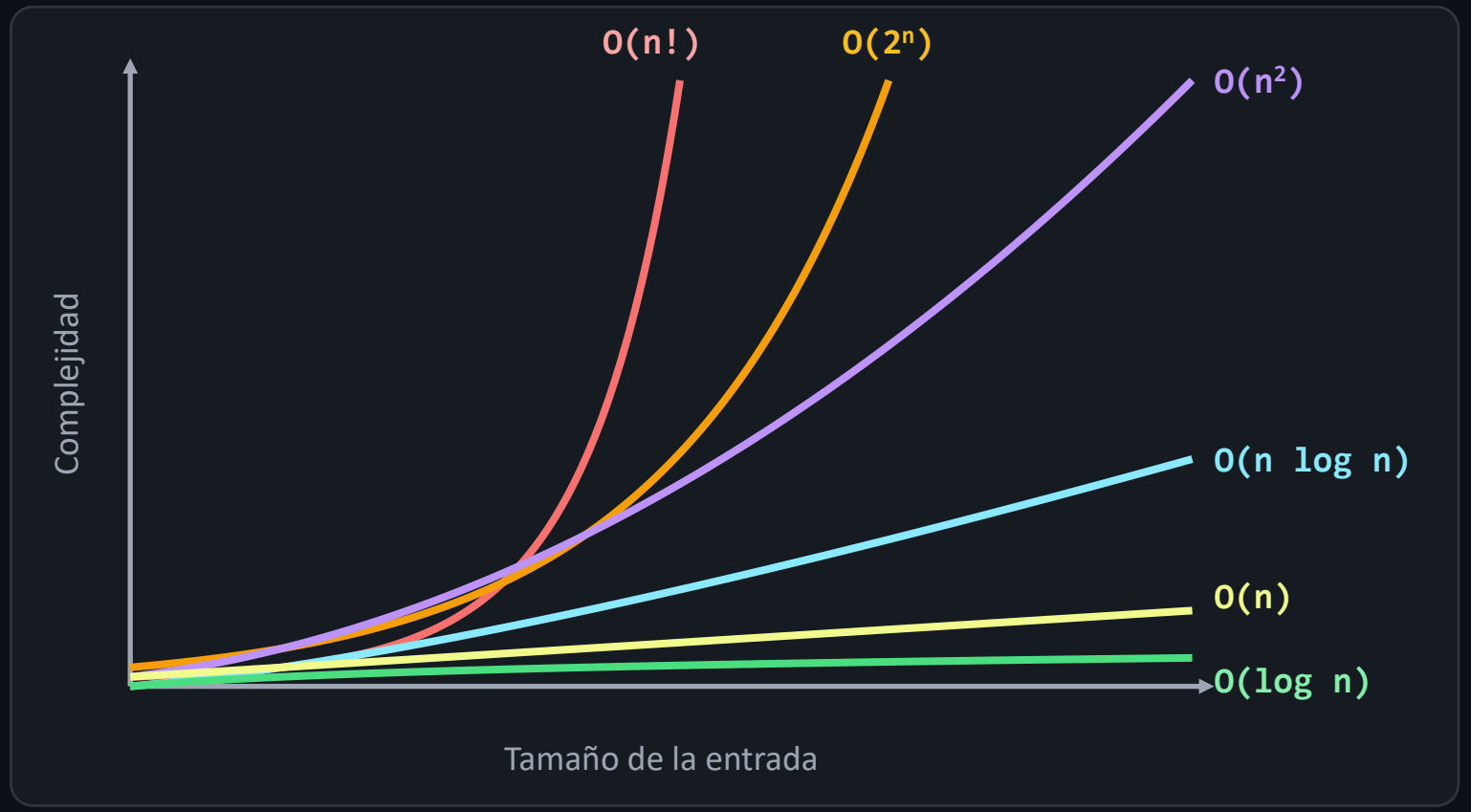
```
int foo(int n) {
    int sum = 0;
    for (int i = 0; i < n; i++) {
        sum += bar(i, 1);
    }
    return sum;
}

int bar(int a, int b) {
    return a + b;
}
```

O(n): se almacenan **n llamadas recursivas** en la pila de llamadas.

O(1): no hay llamadas anidadas: la memoria no crece con n.

Complejidades comunes (Big-O conocidas)



O(n!)
factorial

O(2^n)
exponencial

O(n^2)
cuadrática

O(n log n)
lineal-logarítmica

O(n)
lineal

O(log n)
logarítmica

De menor a mayor crecimiento: logarítmica, lineal, n log n, cuadrática, exponencial y factorial.

// determinar la complejidad

Secuencia de instrucciones e If-Then-Else

Secuencia

```
statement 1;
statement 2;
...
statement n;
```

Se suman: `complejidad(1) + complejidad(2) + ... + complejidad(n)`

If-Then-Else

```
if (condition) {
    statement 1;
} else {
    statement 2;
}
```

Se toma el máximo: `max(complejidad(1), complejidad(2))`

// determinar la complejidad

Loops: simple, secuencial y anidado

Un loop

```
for (i = 0; i < n; i++) {
  statement;
}
```

`n · complejidad(statement)`: `O(n)` si el cuerpo es `O(1)`.

Loops secuenciales

```
for (i = 0; i < n; i++) {
  statement 1;
}
for (j = 0; j < m; j++) {
  statement 2;
}
```

Se suman: la complejidad es **O(n+m)**.

Loops anidados

```
for (i = 0; i < n; i++) {
  for (j = 0; j < m; j++) {
    statement;
  }
}
```

Se multiplican: **O(n^2)** si n = m; con 3 loops, `O(n^3)`.

// determinar la complejidad

Complejidad logarítmica: O(log n)

Código

```
int i = n;
while (i > 0) {
    statement;
    i = i / 2;
}
```

Valor de i en cada iteración

Iteración	Valor de i
1	n
2	n/2
3	n/4
k + 1	n/2^k

El loop termina cuando `n/2^k = 1`, o sea `k = log2(n)`: **cuántas veces se puede dividir n entre 2** hasta llegar a 1.

// determinar la complejidad

Traza con n = 1000: multiplicar vs. dividir entre 2

```
int i = 1;
while (i < n) {
    statement;
    i = i * 2;
}
```

Multiplica por 2: valor de i en cada iteración — **10 iteraciones**

1	2	3	4	5	6	7	8	9	10	sale
1	2	4	8	16	32	64	128	256	512	1024

La siguiente vuelta tendría `i = 1024`: ya no cumple `i < n`.

```
int i = n;
while (i > 0) {
    statement;
    i = i / 2;
}
```

Divide entre 2: valor de i en cada iteración — **10 iteraciones**

1	2	3	4	5	6	7	8	9	10	sale
1000	500	250	125	62	31	15	7	3	1	0

División entera: la siguiente vuelta tendría `i = 0` y el loop termina.

Conclusión: ambos dan unas `log2(1000) ≈ 10` vueltas, pues `2^10 = 1024 ≈ 1000`: son **O(log n)**.

// determinar la complejidad

Recursión: una llamada vs. dos llamadas

Una llamada recursiva

```
int foo(int n) {
    if (n <= 0) {
        return 0;
    }
    return n + foo(n-1);
}
```

Dos llamadas recursivas

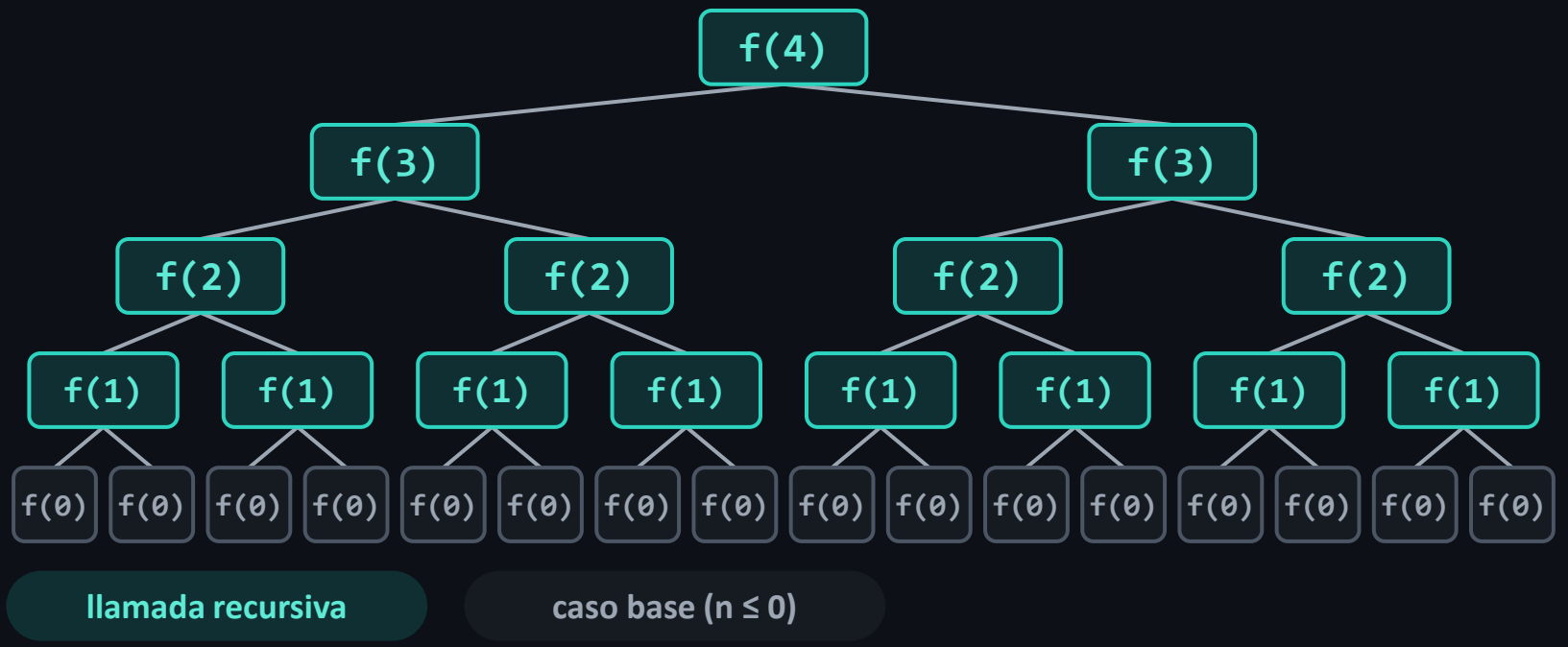
```
int foo(int n) {
    if (n <= 0) {
        return 0;
    }
    return n + foo(n-1) + foo(n-1);
}
```

O(n): la función se llama **n veces**, decrementando n en cada llamada.

O(2^n): cada llamada hace **dos** llamadas: la cantidad se **duplica** por nivel.

// determinar la complejidad

Árbol de llamadas: se duplican por nivel



Nivel	Nodos	Potencia
0	1	2^0
1	2	2^1
2	4	2^2
3	8	2^3
4	16	2^4
Total	**31**	**2^5 - 1**

Hay `2^(n+1) - 1` nodos: la complejidad es **O(2^n)**. En general: `O(ramas^profundidad)`.

// más ejemplos

Las constantes no cambian la complejidad

Ejemplo 1: dos loops sobre el arreglo

```
int sum = 0;
int product = 1;
for (int i = 0; i < array.length; i++) {
    sum += array[i];
}
for (int i = 0; i < array.length; i++) {
    product *= array[i];
}
```

O(2n) = O(n): iterar dos veces el arreglo no importa.

Ejemplo 4: un loop interno constante

```
for (int i = 0; i < a.length; i++) {
  for (int j = 0; j < b.length; j++) {
    for (int k = 0; k < 100000; k++) {
      print(a[i] + "," + b[j]);
    }
  }
}
```

O(ab): a y b son los tamaños de los arreglos; el loop interno es `O(100000)`, una constante.

// más ejemplos

Loops anidados: un arreglo vs. dos arreglos

Ejemplo 2: pares no ordenados

```
for (int i = 0; i < array.length; i++) {
  for (int j = i + 1; j < array.length; j++) {
    print(array[i] + "," + array[j]);
  }
}
```

O(n^2): (N-1) + (N-2) + ... + 1 = N(N-1)/2 operaciones.

Ejemplo 3: dos arreglos

```
for (int i = 0; i < arrayA.length; i++) {
  for (int j = 0; j < arrayB.length; j++) {
    if (arrayA[i] < arrayB[j]) {
      print(arrayA[i] + "," + arrayB[j]);
    }
  }
}
```

O(ab): a es el tamaño de arrayA y b el de arrayB.

// más ejemplos

Recorridos parciales: n/2 y raíz de n

Ejemplo 5: invertir un arreglo

```
for (int i = 0; i < array.length / 2; i++) {
  int other = array.length - i - 1;
  int temp = array[i];
  array[i] = array[other];
  array[other] = temp;
}
```

O(n/2) = O(n): solo se recorre la mitad del arreglo.

Ejemplo 7: ¿es primo?

```
boolean isPrime(int n) {
  for (int x = 2; x <= sqrt(n); x++) {
    if (n % x == 0) {
      return false;
    }
  }
  return true;
}
```

O(sqrt(n)): el loop llega hasta la raíz cuadrada de n.

// más ejemplos

Recursión: árbol, Fibonacci y factorial

Ejemplo 6: sumar un BST

```
int sum(Node node) {
  if (node == null)
    return 0;
  return sum(node.left)
    + node.value
    + sum(node.right);
}
```

Se visita **cada nodo** una vez: $O(n)$.

Ejemplo 8: Fibonacci

```
int fib(int n) {
  if (n <= 0) return 0;
  if (n == 1) return 1;
  return fib(n-1) + fib(n-2);
}
```

Dos llamadas por nivel: $O(2^n)$.

Ejemplo 9: factorial

```
int factorial(int n) {
  if (n < 0) return -1;
  if (n == 0) return 1;
  return n * factorial(n-1);
}
```

Una llamada por nivel: $O(n)$.

Recurrencias: el tiempo de un algoritmo recursivo

Recurrencia: `T(n)` definida con `T` de entradas menores. Ej.: merge sort, `T(n) = 2T(n/2) + O(n)`.

2 T(n/2)

Las **dos llamadas recursivas**, cada una sobre un subproblema de tamaño n/2.

O(n)

El **costo de combinar** las soluciones: el merge, en tiempo lineal.

T(1) = O(1)

El **caso base**: ordenar un solo elemento es trivial.

Resolverla es hallar su **forma cerrada**, con tres métodos: **sustitución**, **árbol de recursión** y **teorema maestro**.

Método de sustitución: adivinar y demostrar

1. Adivinar

Una cota: para `T(n) = 2T(n/2) + n`, `O(n log n)`.

2. Demostrar

Por **inducción matemática**, hallando las constantes.

Paso inductivo (la cota vale para n/2)

```
T(n) =  2 T(n/2) + n
     <= 2 (c * (n/2) * log(n/2)) + n
     =  c * n * log(n/2) + n
     =  c * n * (log(n) - 1) + n
     =  c * n * log(n) - c * n + n
     <= c * n * log(n)
```

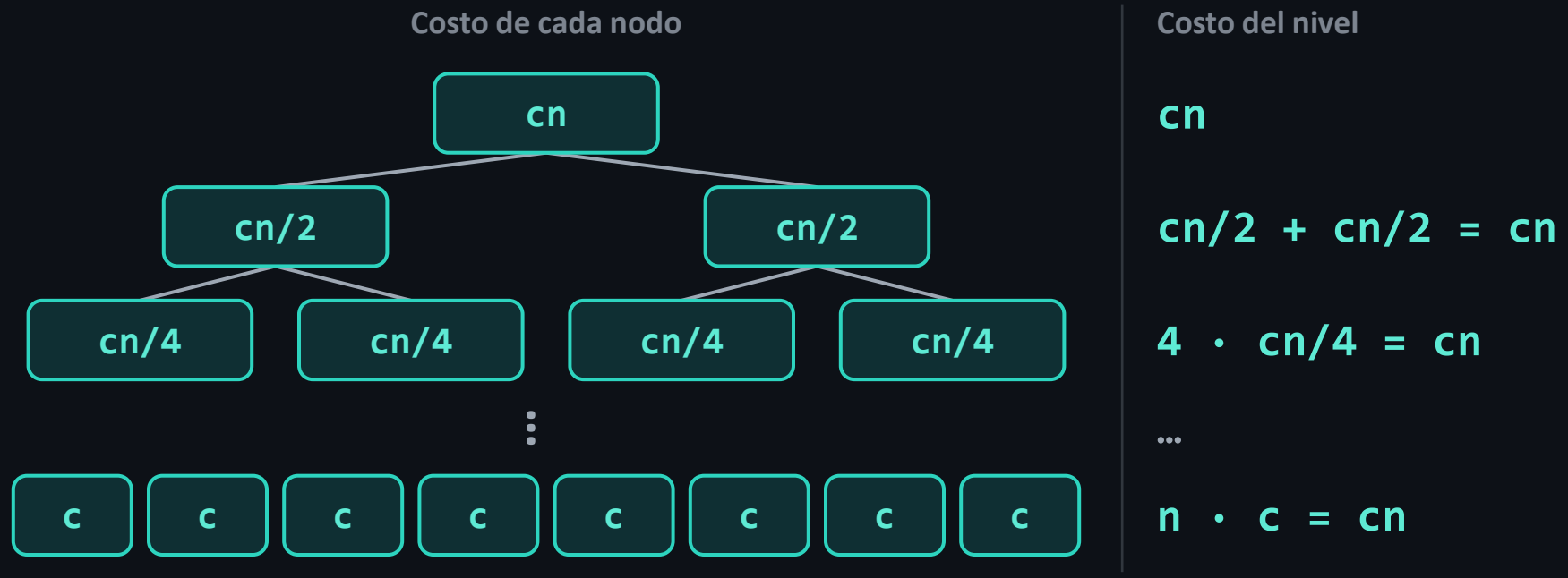
La segunda línea aplica la **hipótesis inductiva**.

Conclusión: con `c >= 1`, `T(n) = O(n log n)`.

Ojo: una mala conjetura (muy holgada o muy ajustada) hace fallar la inducción.

Árbol de recursión: merge sort

Cada nodo es el costo no recursivo `cn` de esa llamada; se suma nivel por nivel.



Hay `log2(n) + 1` niveles: **T(n) = cn · (log2(n) + 1) = O(n log n)**.

// teorema maestro

Teorema maestro: divide y vencerás

Resuelve `T(n) = a * T(n/b) + f(n)`, con `a >= 1` y `b > 1` constantes:

a

Subproblemas

El número de subproblemas en cada paso recursivo.

n/b

Tamaño

El tamaño de cada subproblema: una fracción 1/b del original.

f(n)

Trabajo fuera

El costo fuera de las llamadas: dividir y combinar.

Idea central: **comparar f(n) con n^(log_b a)**, el costo de las hojas. Según cuál domine, hay tres casos.

// teorema maestro

Los tres casos del teorema maestro

Caso	f(n) crece…	Condición (epsilon > 0)	T(n)	Domina
1	más lento	f(n) = O(n^(log_b a - epsilon))	Theta(n^(log_b a))	las hojas
2	igual	f(n) = Theta(n^(log_b a))	Theta(n^(log_b a) · log n)	equilibrio
3	más rápido	f(n) = Omega(n^(log_b a + epsilon))	Theta(f(n))	la raíz

Caso 3: exige la **condición de regularidad**: `a · f(n/b) <= k · f(n)`, con `k < 1`.

No cubre todo: `f(n) = n log n` frente a `n`, ni subproblemas de distinto tamaño.

// teorema maestro

Búsqueda binaria: T(n) = T(n/2) + O(1)

Descarta la mitad del arreglo en cada vuelta

```
int busquedaBinaria(const vector<int>& v, int objetivo) {
    int lo = 0, hi = (int)v.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (v[mid] == objetivo) return mid;
        else if (v[mid] < objetivo) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

1. Identificar

$a = 1, b = 2, f(n) = O(1)$

$n^{(\log_2 1)} = n^0 = 1$

2. Comparar

$f(n) = \Theta(n^0)$: **Caso 2**

$T(n) = \Theta(\log n)$

Cada vuelta descarta la mitad del arreglo: se necesitan unas $\log_2(n)$ vueltas.

// teorema maestro

Más ejemplos con el teorema maestro

Merge sort (Caso 2)

```
T(n) = 2T(n/2) + O(n)
a = 2, b = 2, f(n) = O(n)
n^(log_2 2) = n^1 = n
f(n) = Theta(n)  →  Caso 2
T(n) = Theta(n log n)
```

Coincide con el árbol y la sustitución.

vs

4T(n/2) + O(n) (Caso 1)

```
T(n) = 4T(n/2) + O(n)
a = 4, b = 2, f(n) = O(n)
n^(log_2 4) = n^2
f(n) = O(n^(2-1))  →  Caso 1
T(n) = Theta(n^2)
```

Aquí epsilon = 1.

Resumen: búsqueda binaria, `O(log n)` · merge sort, `O(n log n)` · `4T(n/2) + O(n)`, `O(n^2)`

// práctica

Ahora usted: ¿cuál es la complejidad?

Determine la complejidad de cada caso:

A. Loops anidados

Tres loops anidados: los dos primeros hasta `n` y el tercero hasta `10`.

B. Loop que duplica

```
for (i = 1; i < n; i = i * 2) { ... }
```

C. Recurrencia

`T(n) = 2T(n/2) + n^2`: use el teorema maestro.

D. Recurrencia

`T(n) = 8T(n/2) + n^2`: use el teorema maestro.

Análisis amortizado: tres métodos

Costo **promedio por operación en el peor caso**, sobre una secuencia de m operaciones:

Agregado

Costo total `T(m)` de la secuencia en el peor caso; amortizado: `T(m) / m`.

Contable

Se cobra un costo por operación; el sobrante es **crédito** que paga las caras.

Potencial

Una función `Phi` del estado: costo real más el cambio de potencial.

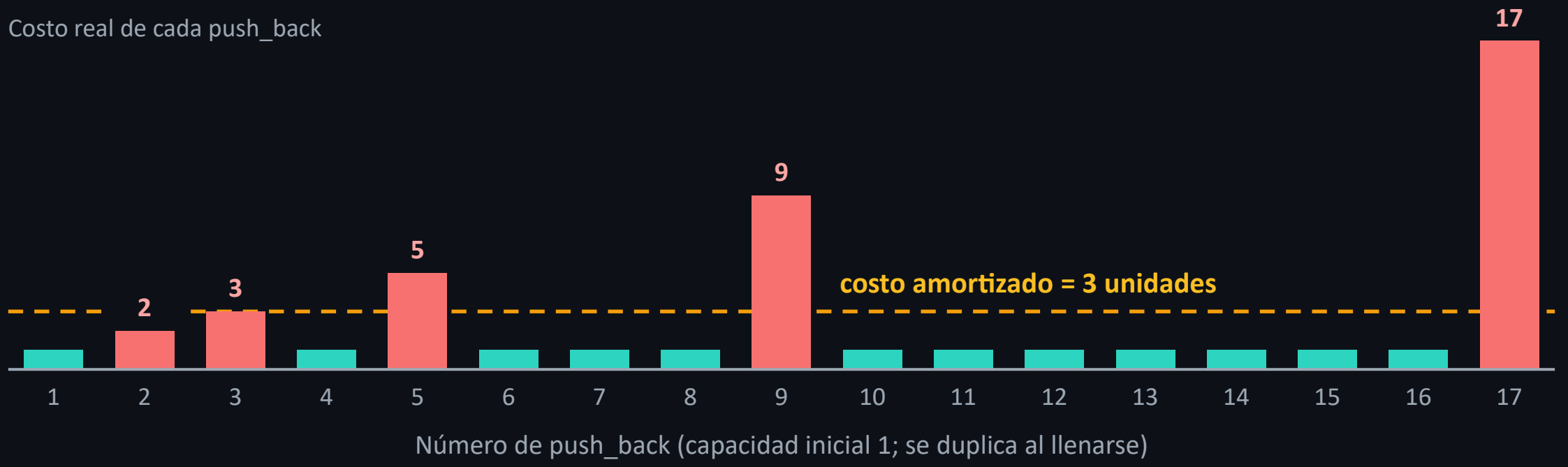
Amortizado ≠ caso promedio: no supone ninguna distribución de probabilidad; es una **garantía** de peor caso.

// análisis amortizado

Arreglo dinámico: casi siempre barato

Inserción simple: costo 1

Redimensión: copiar k elementos + 1



`push_back` parece `O(n)`, pero la redimensión es rara: es **O(1) amortizado**.

// análisis amortizado

Agregado y contable en el arreglo dinámico

Método agregado

Costo total de `m` inserciones:

```
copias <= 1 + 2 + ... + 2^k < 2m
T(m) < m + 2m = 3m = O(m)
T(m) / m = O(1)
```

vs

Método contable

- Se **cobran 3 unidades** por `push_back`
- 1 paga la **inserción** actual
- 1 queda de crédito para copiar **este** elemento
- 1 queda de crédito para copiar **uno antiguo**

Un costo amortizado constante (3) significa que `push_back` es **O(1) amortizado**.

// análisis amortizado

Arreglo dinámico en C++: contando el costo

Clase ArregloDinamico (extracto): copiar es O(n)

```
class ArregloDinamico {
  int* datos;
  size_t tam, capacidad;
  long long costoTotal;
  void redimensionar(size_t nueva) {
    int* nuevos = new int[nueva];
    for (size_t i = 0; i < tam; ++i) {
      nuevos[i] = datos[i]; ++costoTotal;
    }
    delete[] datos; datos = nuevos;
    capacidad = nueva;
  }
```

push_back

```
void push_back(int valor) {
    if (tam == capacidad)
        redimensionar(capacidad * 2);
    datos[tam++] = valor; ++costoTotal;
}
```

Programa de prueba

```
for (int i = 0; i < M; ++i)
    v.push_back(i);
cout << v.costo() / (double)M;
```

Con `M = 1 000 000`, el costo por operación converge a una **constante pequeña** (≈ 2; siempre < 3).

`std::vector` **duplica** su capacidad: si creciera de uno en uno, cada inserción copiaría todo, `O(n)`.

Peor caso de las estructuras de datos (1/2)

Estructura	Acceso	Búsqueda	Inserción	Eliminación
Array	O(1)	O(n)	O(n)	O(n)
Stack	O(n)	O(n)	O(1)	O(1)
Queue	O(n)	O(n)	O(1)	O(1)
Singly linked list	O(n)	O(n)	O(n)	O(n)
Doubly linked list	O(n)	O(n)	O(1)	O(1)

> Todos los valores son de **peor caso**. Solo el arreglo accede en `O(1)`.

Peor caso de las estructuras de datos (2/2)

Estructura	Acceso	Búsqueda	Inserción	Eliminación
Hash table	O(n)	O(n)	O(n)	O(n)
Binary search tree	O(n)	O(n)	O(n)	O(n)
Binary tree	O(n)	O(n)	O(n)	O(n)
AVL tree	O(log n)	O(log n)	O(log n)	O(log n)
Red-black tree	O(log n)	O(log n)	O(log n)	O(log n)

Los árboles **balanceados** (AVL y rojinegro) tienen `O(log n)` en el peor caso.

// cierre

Resumen (1/2): análisis de algoritmos

- **Algoritmo:** procedimiento para cumplir una tarea; el programa es su implementación.
- **Empírico vs. teórico:** medir ejecutando (depende de la máquina) o contar operaciones con un modelo computacional.
- **Casos:** mejor, peor y promedio; el **peor caso** es el más útil.
- **Asintótico:** se descartan constantes y términos menores (domina el de mayor grado); **O**, **Ω** y **Θ** son cotas superior, inferior y ajustada.
- **Complejidad:** secuencia suma, if toma el máximo, loops multiplican; la recursión usa su árbol de llamadas; dividir entre 2 da `O(log n)`.

// cierre

Resumen (2/2): recurrencias y amortizado

- **Recurrencia:** `T(n)` en términos de entradas menores; se resuelve por sustitución, árbol de recursión o teorema maestro.
- **Teorema maestro:** compare `f(n)` con `n^(log_b a)`; tres casos (hojas, equilibrio, raíz) y algunos huecos.
- **Amortizado:** costo promedio por operación en el peor caso sobre una secuencia (agregado, contable y potencial).
- **Arreglo dinámico:** `push_back` es `O(1)` amortizado si se duplica la capacidad.

// cierre

Referencias

- Skiena, S. (2020). The Algorithm Design Manual. Springer.
- Laakmann, G. L. (2020). Cracking the Coding Interview (6th ed.). CareerCup.
- GeeksforGeeks. What is an algorithm and why analysis of it is important
  https://www.geeksforgeeks.org/what-is-algorithm-and-why-analysis-of-it-is-important/